

Chapter 3:

Processes

See class notes

Process Concept

Process scheduling

Operations on processes

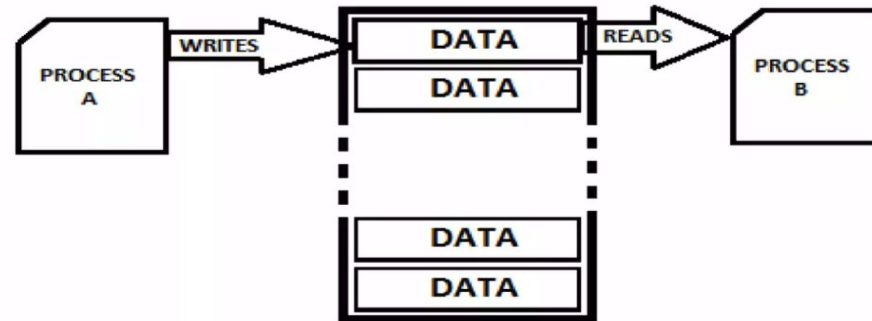
Cooperating processes

Thread

Interprocess communication

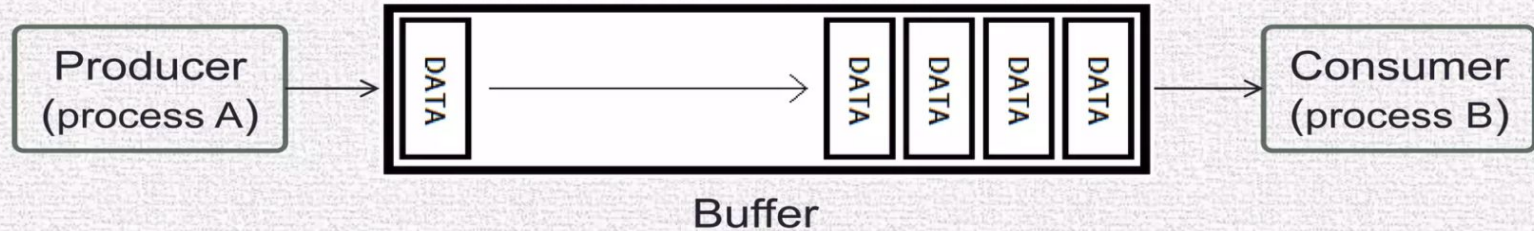
WHAT IS PROCESS SYNCHRONIZATION?

- Several Processes run in an Operating System
- Some of them share resources due to which problems like data inconsistency may arise
- For Example: One process changing the data in a memory location where another process is trying to read the data from the same memory location. It is possible that the data read by the second process will be erroneous



WHAT IS PROCESS SYNCHRONIZATION?

- **PRODUCER CONSUMER PROBLEM**
(or Bounded-Buffer Problem)



- Problem: To ensure that the **Producer** should not add DATA when the Buffer is *full* and the **Consumer** should not take data when the Buffer is *empty*

Why synchronization is needed?

Producer and Consumer run independently, and both access the same buffer. Without proper synchronization:

- Two processes may access the same slot at the same time
- Producer might mistakenly assume space is free when it's not
- Consumer may attempt to read garbage or empty space
- Data might be overwritten or lost
- The system becomes inconsistent or crashes

THE CRITICAL-SECTION PROBLEM

- The critical section problem is the challenge of designing a way so that multiple processes can be cooperate without conflict while shared resources.
- The goal is to prevent **Race Condition**
- A **race condition** happens when two processes (Producer and Consumer) access and modify shared data (like **counter**) at the same time, and the final result depends on the “**Race**” who executes first.

RACE CONDITION

Producer:

```
register1 = counter  
register1 = register1 + 1  
counter = register1
```

Read the counter
Increase it, Write back

Consumer:

```
register2 = counter  
register2 = register2 - 1  
counter = register2
```

Interleaving:

T_0 :	producer	execute	$register_1 = counter$	$\{register_1 = 5\}$
T_1 :	producer	execute	$register_1 = register_1 + 1$	$\{register_1 = 6\}$
T_2 :	consumer	execute	$register_2 = counter$	$\{register_2 = 5\}$
T_3 :	consumer	execute	$register_2 = register_2 - 1$	$\{register_2 = 4\}$
T_4 :	producer	execute	$counter = register_1$	$\{counter = 6\}$
T_5 :	consumer	execute	$counter = register_2$	$\{counter = 4\}$

} ?

CRITICAL SECTION PROBLEM

- **A section of code, common to n cooperating processes, in which the processes may be accessing common variables.**
- A Critical Section Environment contains:
- **Entry Section** Code requesting entry into the critical section.
- **Critical Section** Code in which only one process can execute at any one time.
- **Exit Section** The end of the critical section, releasing or allowing others in.
- **Remainder Section** Rest of the code AFTER the critical section.

CRITICAL SECTION PROBLEM

```
do {
```

```
    while ( turn ^= i );
```

```
    /* critical section */
```

```
    turn = j;
```

```
    /* remainder section */
```

```
} while(TRUE);
```

ENTRY SECTION

CRITICAL SECTION

EXIT SECTION

REMAINDER SECTION

CRITICAL SECTION PROBLEM

- The critical section must **ENFORCE ALL THREE** of the following rules:
- **Mutual Exclusion:** No more than one process can execute in its critical section at one time.
- **Progress:** If no one is in the critical section and someone wants in, then those processes not in their remainder section must be able to decide in a finite time who should go in. Who goes next without delay
- **Bounded Wait:** All requesters must eventually be let into the critical section Process must not wait forever

Peterson's Solution

- A classical **software based** solution
- May not work correctly on **modern computer architecture**
- However, it provides a **good algorithmic description**
- Address the requirements of **Mutual Exclusion, Progress and Bounded Wait**
- Restricted to Two Processes: P_i and P_j

It requires two data items to be shared between the two processes:

Shared Data Items

`int turn`

- Indicates whose turn it is to enter the critical section

`boolean flag[2]`

- Indicate if a process is ready to enter it's critical section

PETERSON'S SOLUTION

- To handle the problem of Critical Section (CS), Peterson gave an algorithm with a bounded waiting
- Suppose there are N processes ($P_1, P_2, \dots, P_N$) and each of them at some point need to enter the Critical Section
- A `FLAG[]` array of size N is maintained which is by default false and whenever a process need to enter the critical section it has to set its flag as true, i.e. suppose P_i wants to enter so it will set `FLAG[i]=TRUE`
- There is another variable called `TURN` which indicates the process number which is currently to enter into the CS. The process that enters into the CS while exiting would change the `TURN` to another number from among the list of ready processes

Structure of process P_i

do {

```
flag [ i ] = true;    // Ready to enter it's critical section  
turn = j;             // Asking other process if it wants to enter it's C.S  
while ( flag[ j ] && turn == j );
```

Critical Section

```
flag[ i ] = false;
```

Remainder Section

} while(true);

Process P_i

do {

```
flag[ i ] = true;
```

```
turn = j;
```

```
while ( flag[ j ] && turn == j );
```

Critical Section

```
flag[ i ] = false;
```

Remainder Section

} while(true);

Process P_j

do {

```
flag[ j ] = true;
```

```
turn = i;
```

```
while ( flag[ i ] && turn == i );
```

Critical Section

```
flag[ j ] = false;
```

Remainder Section

} while(true);

Break down example:

```
flag[i] = true;
```

```
turn = j;
```

```
while (flag[j] && turn == j);
```

- P_i sets its flag = “I want to enter the critical section.”
- P_i gives priority to P_j by setting $turn = j$.
- If P_j is **not interested**, $flag[j] == false$, so the while loop becomes:

```
while (false && turn == j) → false
```